Event Management System – Full Technical Documentation

1. Project Overview

This documentation describes the complete architecture, flow, database structure, backend API design, frontend design, ticketing system, payment process, and deployment plan for building a Wix-Events–style Event Management System using **Option A: MERN-style architecture (React + Node.js + PostgreSQL)**.

The documentation is written so it can be directly used by GitHub Copilot to scaffold and implement the project.

---

2. System Features

Admin Portal Features

- Admin login & authentication
- Create, update, delete events
- Upload event posters & media
- Publish/unpublish events
- Create custom registration forms for each event
- View registrations and payments
- Download/export attendee list
- View analytics (no. of tickets sold, revenue, etc.)

Public User Features

- View/browse published events
- View event details and poster
- Fill the registration form defined by admin
- Redirect to payment gateway
- Receive a **QR ticket PDF via email** after successful payment

Additional System Features

- HMAC-signed QR codes for secure verification
- Webhook-based payment confirmation
- Background job for generating PDF tickets
- Ticket verification API for scanning at venue

---

3. Tech Stack

Frontend

- React (Vite or CRA)
- Tailwind CSS
- Axios for API calls
- React Router

Backend

- Node.js + Express
- PostgreSQL (via Prisma ORM or Knex)
- JSON schema validation (AJV)
- Nodemailer for emails
- QR code generator: `qrcode`
- PDF generator: Puppeteer (HTML → PDF)
- File storage: AWS S3 or local storage during development

Payments

- Razorpay or Stripe
- Webhook verification

Optional Worker Queue

- Redis + BullMQ for ticket generation and email sending

---

4. High-Level Architecture

```
Client (React)
    ↓ via REST API
Backend (Node + Express)
    ↓
PostgreSQL DB
    ↓
Cloud Storage (S3) for posters + PDFs
    ↓
Payment Gateway (Razorpay/Stripe)
    ↓
Webhook → Backend → Background Worker
```

---

5. Database Schema (PostgreSQL)

Users Table

```
users
- id (uuid)
- name
- email
- password_hash
- role ("admin" | "organizer")
- created_at
```

Events Table

```
events
- id (uuid)
- organizer_id (fk → users.id)
- title
- slug
- description
- location
- start_time
- end_time
- capacity
- price_cents
- currency
- poster_url
- published (bool)
- created_at
- updated_at
```

Forms Table

```
forms
- id (uuid)
- event_id (fk → events.id)
- schema_json (jsonb)  // form builder output
- created_at
```

Registrations Table

```
registrations
- id (uuid)
- event_id
- user_email
- form_response (jsonb)
```

```
- status ("pending" | "paid" | "cancelled")
- created_at
```

Orders Table

```
orders
- id (uuid)
- registration_id (fk → registrations.id)
- amount_cents
- currency
- provider ("stripe" | "razorpay")
- provider_order_id
- status ("created" | "paid" | "failed")
- payment_data (jsonb)
- created_at
- updated_at
```

Tickets Table

```
tickets
- id (uuid)
- order_id
- qr_payload (string)
- ticket_pdf_url
- issued_at
- revoked (bool)
- valid_until
```

---

6. Custom Registration Form Architecture

Admin uses a UI to define a form. The form is stored as `schema_json`:

```
{
  "title": "Attendee Information",
  "fields": [
    {"key": "name", "type": "text", "label": "Full Name", "required": true},
    {"key": "email", "type": "email", "label": "Email", "required": true},
    {"key": "phone", "type": "tel", "label": "Phone", "required": false},
    {"key": "gender", "type": "select", "label": "Gender", "options":
["M","F","Other"], "required": false}
  ]
}
```

Frontend renders dynamically. Backend validates using AJV.

---

7. REST API Specification

Auth

```
POST /api/auth/login
POST /api/auth/register
```

Public Event Endpoints

```
GET /api/events
GET /api/events/:id
```

Admin Event Management

```
POST /api/admin/events
PUT /api/admin/events/:id
DELETE /api/admin/events/:id
POST /api/admin/events/:id/poster-upload
```

Form Builder

```
POST /api/admin/events/:id/form
GET /api/events/:id/form
```

Registration + Payment

```
POST /api/events/:id/register
POST /api/orders/:id/create-checkout-session
POST /api/webhooks/payments
```

Tickets

```
GET /api/tickets/:id/pdf   // authorized or signed download link
POST /api/tickets/verify   // for scanning at the venue
```

---

8. Payment Flow

1. User submits registration form.
2. Backend creates:
3. `registration` row
4. `order` row
5. Backend creates payment session using Stripe/Razorpay.

6. Frontend redirects user to payment gateway.
7. Payment gateway calls backend webhook.
8. Backend:
9. verifies signature
10. marks order as `paid`
11. marks registration `paid`
12. creates ticket entry
13. triggers PDF + email job
14. User receives email with QR-ticket PDF.

---

9. Ticket Generation (QR + PDF)

QR Payload Structure

```
{
  "ticketId": "uuid",
  "eventId": "uuid",
  "issuedAt": 1731600000,
  "sig": "HMAC-SHA256 signature"
}
```

Steps to Generate Ticket

- Generate QR code using `qrcode` library.
- Render HTML ticket template.
- Convert to PDF using Puppeteer.
- Upload to S3.
- Update tickets table.
- Send email to user with PDF.

---

10. Email Delivery

Use **Nodemailer** with SendGrid/Mailgun/SES.

Email contains: - Event details - Ticket ID - Attached PDF OR download link - QR image inline

---

11. Admin Panel Frontend Structure (React)

Pages

- Login
- Dashboard
- Create Event
- Edit Event
- Event List

- Form Builder
- Registrations List
- Ticket Download/Preview

Components

- EventCard
- FormBuilder
- DynamicFormRenderer
- ImageUploader
- QRPreview

---

12. Public Frontend Structure (React)

Pages

- Home (list events)
- Event Details
- Registration Form
- Payment Redirect
- Success Page

---

13. Ticket Verification Flow

Venue staff scans QR → scanner app sends QR payload to:

```
POST /api/tickets/verify
```

Server checks: - Signature valid? - Ticket exists? - Not expired? - Not previously used? If OK → mark ticket used.

---

14. Background Worker (Optional but recommended)

Use **Redis + BullMQ** for queues: - `ticket_generation_queue` - `email_queue`

Flow: Backend webhook → enqueue job → Worker processes → updates DB.

---

15. Security Best Practices

- HTTPS required
- Use JWT or session cookies
- Validate all user inputs server-side
- HMAC signing for QR security
- Verify payment gateway signatures

- Don’t store card data
- Limit ticket download to owner
- Rate-limit login & registration

---

16. Deployment Plan

Frontend

- Deploy to Vercel/Netlify/S3 + CloudFront

Backend

- Deploy to EC2 / Render / Railway / DigitalOcean
- Use PM2 or Docker

Database

- Use managed PostgreSQL (Neon, Supabase, AWS RDS)

Storage

- Use AWS S3 (recommended)

Worker

- Deploy as separate Node service

---

17. MVP Development Roadmap

1. Authentication system
2. Event CRUD APIs
3. Poster upload
4. Form builder
5. Registration & order creation
6. Payment integration
7. Webhook handler
8. Ticket & PDF generation
9. Email integration
10. Public UI + Admin UI polish
11. Scanner app + verification API

---

18. Conclusion

This documentation fully defines the structure, architecture, workflow, and implementation methodology for building a Wix-style Event Management System using Node.js, React, and PostgreSQL.

You can now provide this directly to **GitHub Copilot**, and it will be able to generate the necessary code files, components, APIs, and infrastructure.